TheGrue - HackOn CTF Writeup

Challenge: TheGrue
Category: Misc / GamePwn
Points: 387
Author: Znati
Platform: HackOn
Flag: `HackOn{Th3_Gru3_Gr4du4d0!!}`

Descripción

> The Grue es un monstruo legendario, famoso por habitar la oscuridad absoluta y devorar a los aventureros.
>
> Nos hemos quedado sin electricidad en la URJC y he perdido a mis amigos, es peligroso estar por ahí sin luz. Tengo que hacer todo lo posible para encontrarlos antes de que lo haga The Grue o podría pasar lo peor.

Se nos proporciona un archivo `TheGrue.zip` que contiene un juego desarrollado en Unity.

Reconocimiento

Identificación del engine y backend

Al descomprimir el archivo, la estructura del juego revela que se trata de un proyecto Unity compilado con el backend **IL2CPP**:

```
TheGrue/
├── TheGrue.exe
├── GameAssembly.dll          ← Binario nativo (lógica del juego compilada)
├── TheGrue_Data/
│   └── il2cpp_data/
│       └── Metadata/
│           └── global-metadata.dat  ← Metadata con tipos, strings y offsets
```

La presencia de `GameAssembly.dll` en lugar de `Assembly-CSharp.dll` confirma que el juego usa **IL2CPP** en vez de Mono. Esto significa que el código C# original ha sido compilado a

código nativo (C++), lo que dificulta el reversing directo, pero la metadata sigue conteniendo la información necesaria para reconstruir las estructuras.

Extracción de metadata con Il2CppDumper

Utilizamos Il2CppDumper para extraer la información de tipos, métodos y strings:

```
.\Il2CppDumper.exe ..\GameAssembly.dll `
    ..\TheGrue_Data\il2cpp_data\Metadata\global-metadata.dat output/
```

```
Initializing metadata...
Metadata Version: 31
Initializing il2cpp file...
Il2Cpp Version: 31
Searching...
Change il2cpp version to: 29
CodeRegistration : 181035060
MetadataRegistration : 181226650
Dumping...
Done!
```

Il2CppDumper genera varios archivos clave:

- `dump.cs` — Declaraciones de todas las clases con sus offsets (RVA).
- `stringliteral.json` — Strings hardcodeadas del juego.
- `script.json` — Para importar en Ghidra/IDA con nombres resueltos.

Análisis estático

Namespaces del juego

Filtrando los namespaces personalizados en `dump.cs`:

```
grep -n "// Namespace:" dump.cs | grep -i "Gamepwn"
```

Encontramos la arquitectura del juego bajo el namespace **Gamepwn**:

Namespace	Clases
`Gamepwn.Core`	GameManager, InventorySystem, ItemDatabase, MoneyManager, Item, Interactable
`Gamepwn.Player`	PlayerController
`Gamepwn.Environment`	DavidNPC, Door, GrueTrigger, Pickup, VendingMachine, SignInteractable, TVInteractable
`Gamepwn.UI`	GameLogger, InventoryUI

VendingMachine — Vector de ataque

```
// Namespace: Gamepwn.Environment
public class VendingMachine : Interactable
{
    [Header("Vulnerability A Logic")]    // ← Pista del autor
    public int selectedItemID;            // 0x20
    private int _transactionPrice;        // 0x24
    private bool _isPlayerInRange;        // 0x28

    // RVA: 0x2978C0
    public void BuyInput() { }

    // RVA: 0x297EC0
    public void SelectItem(int itemId) { }
}
```

El header `"Vulnerability A Logic"` es una pista directa del autor indicando que la máquina expendedora contiene una vulnerabilidad explotable.

MoneyManager — Dinero ofuscado

```
public class MoneyManager : MonoBehaviour
{
    private ObfuscatedInt _currentMoney;   // 0x18 — Valor ofuscado en memoria

    // RVA: 0x294E80
    public int GetMoney() { }

    // RVA: 0x2950F0
    public bool TrySpendMoney(int amount) { }

    // RVA: 0x2951F0
    public void UpdateReserves(int amount, int checksum) { }
}
```

El dinero está protegido mediante `ObfuscatedInt`, lo que significa que el valor real en memoria está cifrado con XOR y no se puede modificar directamente con un memory scan convencional.

Strings relevantes

Del `stringliteral.json` extraemos los mensajes clave:

```
"¡CUIDADO! No entres en habitaciones oscuras sin luz, puedes ser devorado por El
Grue."
"Parece que lograste engañar al Grue, pero sin luz no encontrarás nada..."
"Tienes la linterna! El Grue se asusta y escapa!."
"[Dr3y]: Gracias! Estaba escuchando al Grue acercándose..."
"YOU WIN!"
"flag"
```

Flujo del juego

Con toda esta información, reconstruimos el flujo de victoria:

```
Recoger dinero (Pickup) → Comprar linterna en VendingMachine →
Entrar en habitación oscura (GrueTrigger) → Rescatar a Dr3y (DavidNPC) →
GameManager.WinGame() → FLAG
```

El problema es que el jugador no tiene suficiente dinero para comprar la linterna (precio: 7€), lo que nos obliga a explotar la vulnerabilidad de la máquina expendedora.

Análisis dinámico con Cheat Engine

Desensamblado de BuyInput

Navegando a `GameAssembly.dll+2978C0`, el desensamblado revela el flujo de compra con la verificación de dinero:

```
; --- Desofuscar dinero (ObfuscatedInt) ---
mov ecx, [rbx+18]                    ; valor cifrado A
mov eax, [rbx+1C]                    ; valor cifrado B
xor ecx, eax                         ; dinero_real = A XOR B

; --- Comparación crítica ---
cmp ecx, ebp                         ; dinero_real vs precio
jl  297BFA                            ; ★ si dinero < precio → "No tienes dinero"
```

La línea crítica es:

```
2979B5: 3B CD     ; cmp ecx, ebp  → compara dinero (ecx) con precio (ebp)
2979B7: 0F 8C ... ; jl 297BFA     → salta si dinero < precio
```

Anti-tamper CAFEBABE

Además del check de dinero, hay una verificación de integridad que detecta si el precio ha sido manipulado en memoria:

```
297A62: imul rsi, rbp, 5F3759DF    ; hash del precio
297A69: mov  eax, CAFEBABE          ; constante mágica
297A6E: xor  rsi, rax               ; verificación
297A71: je   297BFA                  ; si match → misma ruta de error
```

Explotación

Estrategia: Parche de 1 byte

En lugar de parchear con NOPs (que rompe el flujo de registros y causa crashes), aplicamos un parche directo de **un solo byte** que cambia el operando de la comparación:

```
Antes:  3B CD    → cmp ecx, ebp   (dinero vs precio)
Después: 3B ED   → cmp ebp, ebp   (precio vs precio — siempre igual)
```

Al comparar `ebp` consigo mismo, el resultado siempre es "igual", por lo que el `jl` (jump if less) nunca se toma y la compra siempre procede. El parche no modifica datos en memoria, evitando así la protección anti-tamper CAFEBABE.

Aplicación del parche con Lua

Utilizamos la consola Lua de Cheat Engine para aplicar el parche de forma programática:

```lua
local base = getAddress("GameAssembly.dll")

-- Parche: cmp ecx,ebp → cmp ebp,ebp (1 byte: CD → ED)
local addr = base + 0x2979B6
writeBytes(addr, 0xED)

print("Patch applied - buy check bypassed")
```

Al ejecutar el script y pulsar el botón de compra en el juego, la linterna se añade al inventario con éxito:

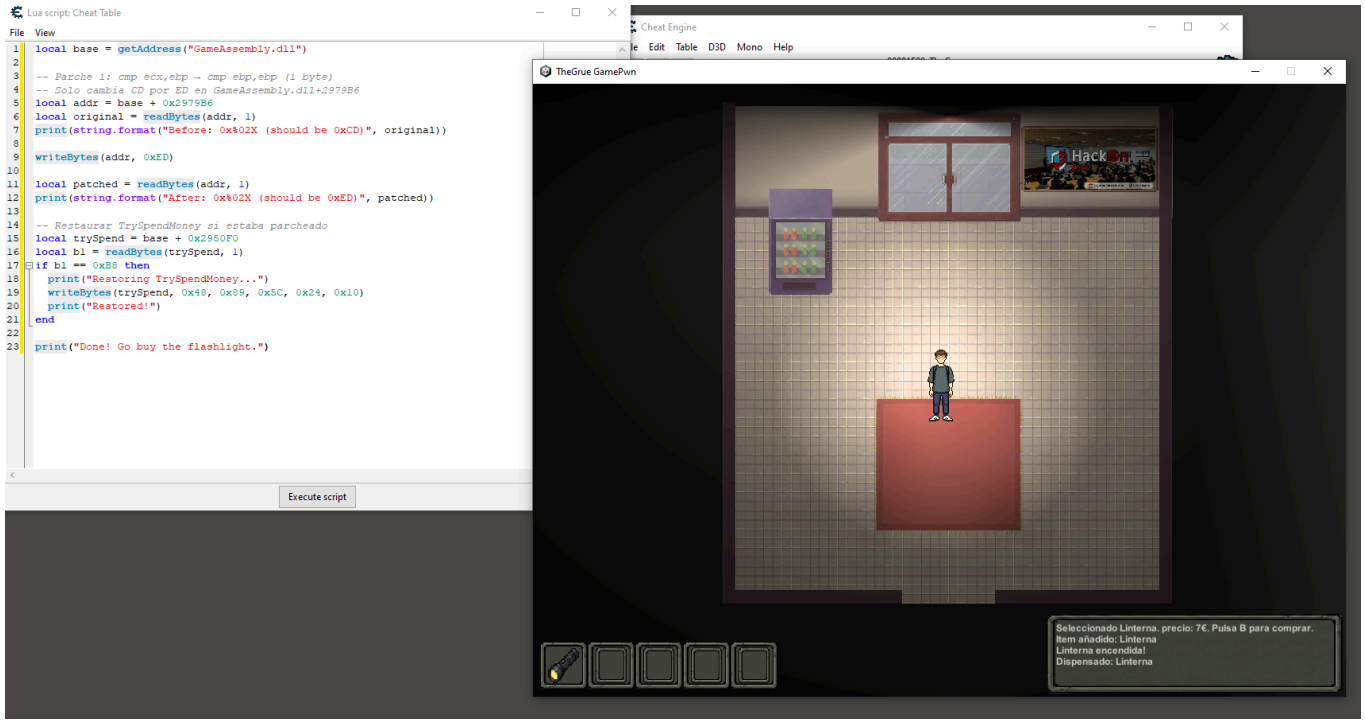

Figura 1 — Script Lua aplicado en Cheat Engine. El log del juego muestra: "Item añadido: Linterna / Linterna encendida! / Dispensado: Linterna"

Obtención de la flag

Con la linterna equipada, el jugador puede entrar en la habitación oscura sin ser devorado por el Grue. Dentro, encontramos al NPC **Dr3y** que nos entrega la flag al interactuar con él:

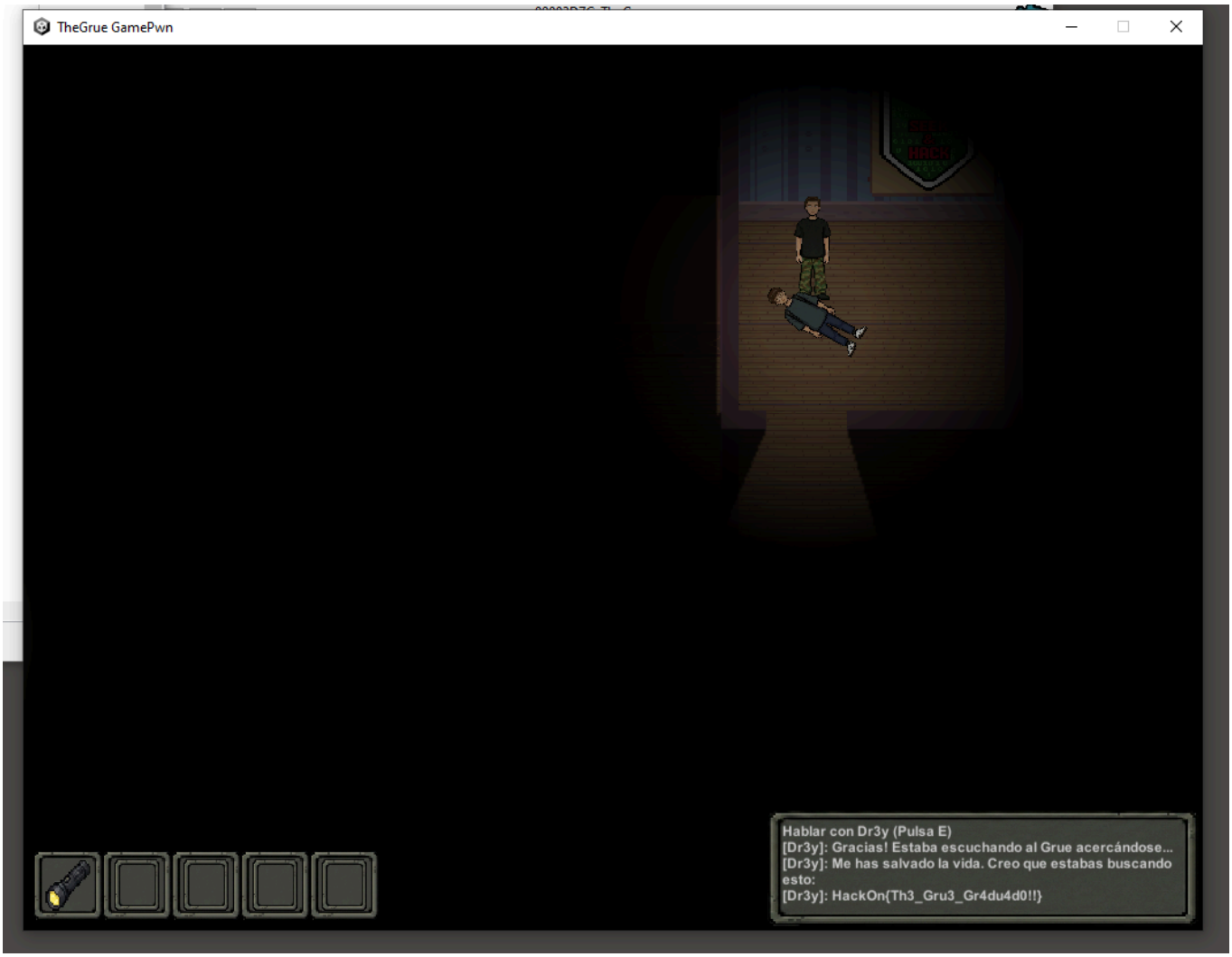

Figura 2 — Dr3y rescatado en la habitación oscura. Diálogo: "[Dr3y]: Gracias! Estaba escuchando al Grue acercándose... Me has salvado la vida. Creo que estabas buscando esto: HackOn{Th3_Gru3_Gr4du4d0!!}"

Verificación en memoria

Tras completar el reto, la flag se puede confirmar buscando en memoria del proceso con un AOBScan en la consola Lua de Cheat Engine:

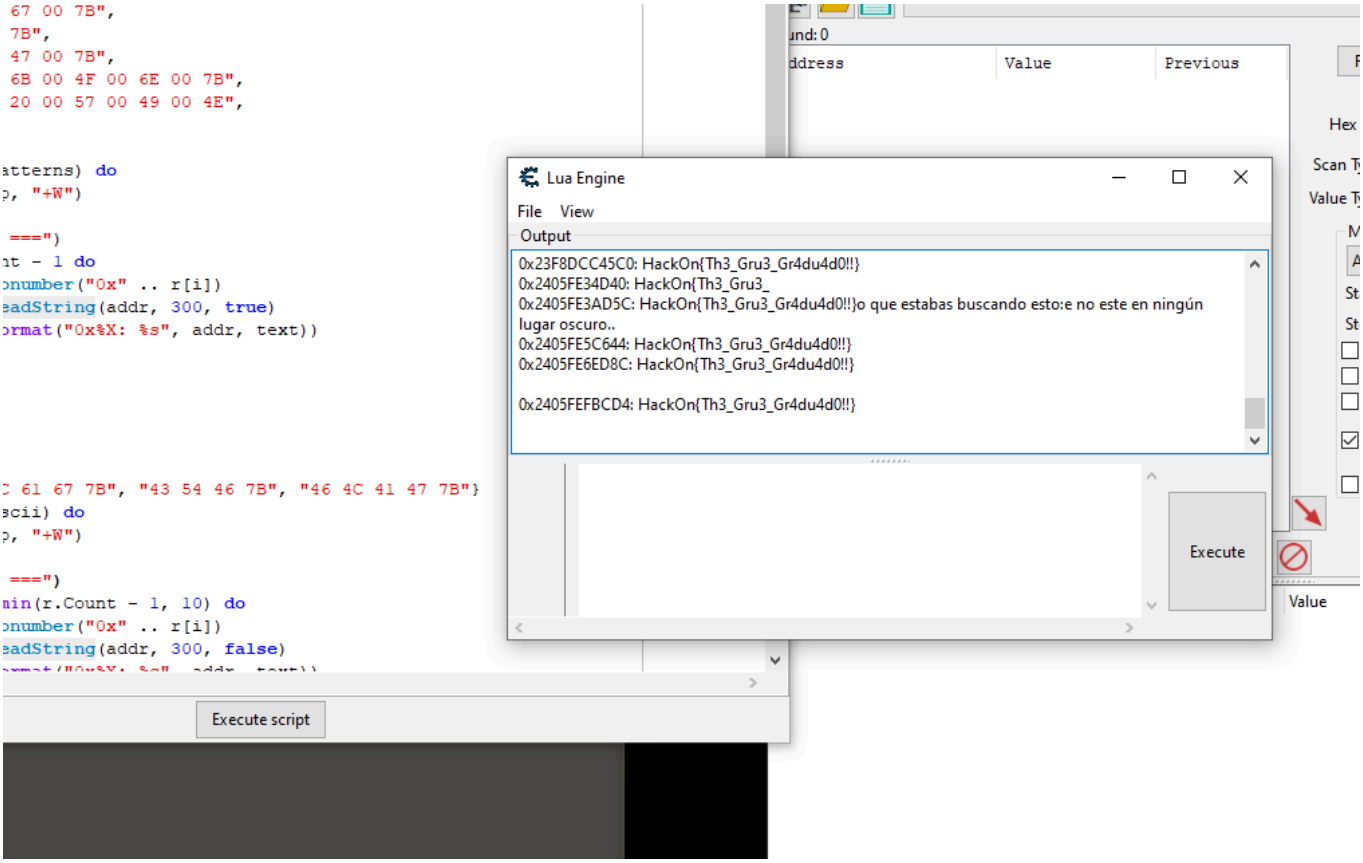

Figura 3 — La flag `HackOn{Th3_Gru3_Gr4du4d0!!}` aparece en múltiples ubicaciones de memoria tras completar el juego

---

Flag

```
HackOn{Th3_Gru3_Gr4du4d0!!}
```

Herramientas utilizadas

Herramienta	Uso
Il2CppDumper	Extracción de metadata y generación de dump.cs con RVAs
Cheat Engine	Análisis dinámico, breakpoints, memory scan y patching en runtime
PowerShell	Búsqueda de strings y análisis de archivos de metadata

Resumen de técnicas

- **Reversing de IL2CPP**: Uso de Il2CppDumper para recuperar la estructura de clases, métodos y RVAs desde un binario nativo Unity.
- **Análisis de ObfuscatedInt**: Identificación del esquema de ofuscación XOR ($A \oplus B$ = valor real) para el sistema de dinero del juego.
- **Parche de 1 byte**: Modificación del operando de la instrucción `cmp` (`CD` → `ED`) para alterar la comparación sin romper el flujo de ejecución ni activar el anti-tamper CAFEBABE.
- **Memory scanning con AOBScan**: Búsqueda de strings UTF-16LE para localizar objetos Item en memoria y confirmar sus propiedades.
- **Anti-tamper bypass**: Evasión de la protección CAFEBABE mediante un enfoque que modifica la instrucción de comparación en lugar de los datos en memoria.